ACTIA®
ENGINEERING SERVICES

Embedded Team Academy

Advanced C++ training

Prepared and Presented by
Mahdi SAKKA

ACTIA®
ENGINEERING SERVICES

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Agenda

Module 1: C++ Basics

Module 2: C++ Object Oriented Programming

Module 3: C++ Standard Library

Module 4: C++ Object Oriented Programming

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Agenda

Module 3: C++ Standard Library

- Containers
- Iterators
- Smart pointers
- Algorithms
- String
- Date and time management
- Multi-threading and threads synchronization
- Exceptions
- Files management
- Regular expressions

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Standard Library

- One of the advantages of C++ is standard library
- It was written in optimized way to provide many common features that help developing programs without defining them and wasting time
- No need to include all the standard library, just choose the features that we need
- Always refer to documentation for further details https://en.cppreference.com

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

Containers

- Two major types of containers

Sequence Containers	Associative Containers
std::array	std::set
std::vector	std::map
std::deque	std::multiset
std::list	std::multimap
std::forward_list	std::unordered_set
std::stack	std::unordered_map
std::queue	std::unordered_multiset
std::priority_queue	std::unordered_multimap

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Sequence Containers

std::array is a static (allocation) contiguous array, the size should be fixed at declaration

std::array<class_name, size> array_object{obj1, obj2, ... }

Method	Explanation
at(std::size_t x)	Gets the value at position x by reference with bound checking (may throw exception if out of range)
operator[](std::size_t x)	Gets the value at position x by reference without bound checking
front()	Returns reference to first element
back()	Returns reference to last element
size()	Returns the size of the array (number of elements that it holds)
empty()	Return true if no values in the array
fill(const Type&)	Fill the total array with value

☐ Provisoire ☐ Public ☐ Interne ☑ Confidentiel ☐ Secret

ACTIA®

Sequence Containers

- **std::vector** is a dynamic (allocation) contiguous array, the size should be fixed at declaration

```
// with initialization

std::vector<class_name> vector_object {obj1, obj2, .... };

// without initialization

std::vector<class_name> vector_object;
```

- Refer to documentation for detailed features

https://en.cppreference.com/w/cpp/container/vector

Sequence Containers

- **std::vector**
  - vector capacity is the dynamically allocated memory
  - vector size is the number of elements in the vector
  - we can act on capacity by methods '*reserve(x)*' and '*shrink_to_fit()*'
  - we can act on size by methods '*emplace*', *emplace_back*', '*push_back*', '*pop_back*', '*clear*', '*erase*', '*resize*', '*insert*'

Sequence Containers

- **std::deque** double ended queue: used for fast insertion and deletion of elements at both ends of queue (this is the advantage of deque)
- Memory is not contiguous (unlike vector and array) and no capacity concept
- But can do almost same operations like vector (insertion, deletion at the middle, ...)

```
// with initialization

std::deque<class_name> deque_object {obj1, obj2, .... };

// without initialization

std::deque<class_name> deque_object;
```

Provisoire Public Interne Confidentiel Secret

ACTIA®

Sequence Containers

- **std::list** a container that allows non contiguous memory allocation.
- *Point of strongness: insertion and deletion are very quick if insertion position is found*
- *Point of weakness: the list has to be parsed entirely to find the wanted element*
- *Iteration is bidirectional (will be discovered in iterators section)*

Sequence Containers

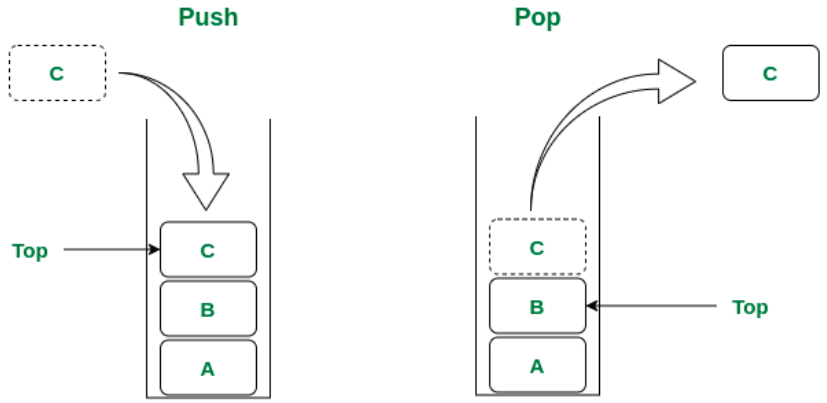
- **std::forward_list** a container that allows non contiguous memory allocation.
- *Point of strongness: supports fast insertion and removal of elements from anywhere in the container*
- *Point of weakness: the list has to be parsed entirely to find the wanted element*
- *Iteration is unidirectional (but memory space efficient compared to lists)*
- *Operations can be done only from front ('push_front', 'pop_front')*

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

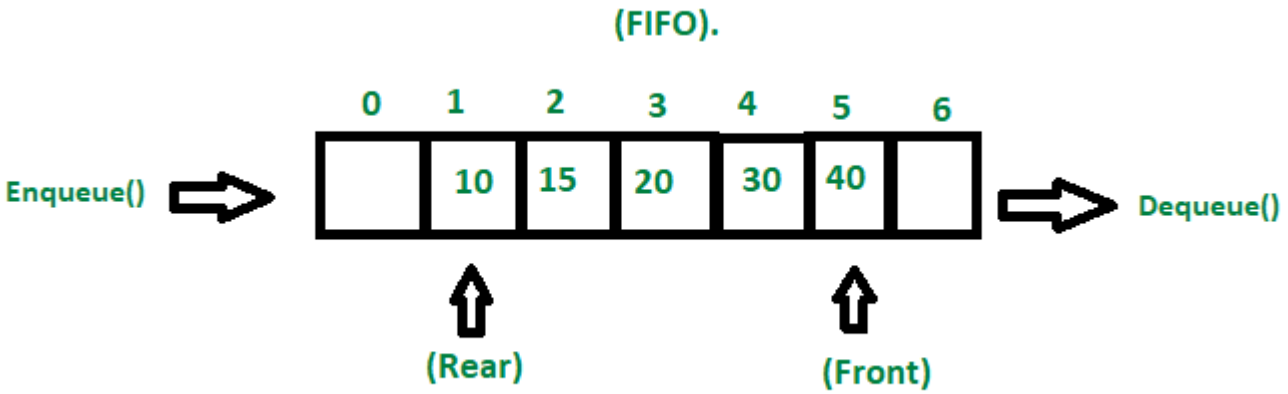
Sequence Containers

- **std::stack** (Last In First Out) the so called LIFO stack
- Elements are added and removed only from top
- Stack cannot be parsed, we can only access element on top



Sequence Containers

- **std::queue** (First In First Out) the so called FIFO stack
- Elements are added from one end and accessed from the other end
- Stack cannot be parsed, we can only access element on top



Sequence Containers

- **std::priority_queue** is a queue that guarantees that the first element is always the greatest or smallest element
- By default, the greatest element is the first element
- Only the first element can be accessed
- The type held by priority_queue must implement the 'operator<' or define its own comparer

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Associative Containers

- **std::set** is an associative container that stores element (keys) so that they will be unique because the value of element identifies it.
- Elements are always sorted in specific order
- Default order is ascending (first element is the lowest)
- Comparer has to be provided to change the sorting order

Associative Containers

- **std::map** is an associative container that holds key-value pairs
- Pairs are stored in sorted way, default is ascending order
- Comparison is done on keys for ordering pairs
- Keys are unique within each map (using an already existing key will override the corresponding value
- Using the operator [] creates a pair if it does not exit, so the value existence has not to be checked using this operator

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Associative Containers

- **std::multiset** is an associative container that holds sorted elements (keys)
- Unlike set, multiset can hold duplicate keys
- Default sorting order is ascending
- Same as set, key object needs to have comparison operator

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Associative Containers

- **std::multimap** is an associative container that holds key-value pairs
- Pairs are stored in sorted way, default is ascending order
- Comparison is done on keys for ordering pairs
- Multimap allows duplication of keys
- Using the operator [] and ‘at()’ function is not allowed unlike map

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Associative Containers

- **std::unordered_set** is an associative container that contains element that doesn't follow specific order
- Element should be unique -> it uses a hash algorithm for storage to allow fast access to elements
- Elements cannot be modified (change value) to not corrupt the container
- Objects that are stored in unordered_set need to overload 'operator=='

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Associative Containers

- **std::unordered_map** is an associative container that stores key value pairs without sorting
- Must define a template specialization for std::hash with specific 'key' type if the 'key' type is not native type
- Element keys should be unique -> it uses a hash algorithm for storage to allow fast access to elements
- Objects that are stored in unordered_set need to overload 'operator=='
- The 'operator[]' and 'at()' function are available for unordered_map

Associative Containers

- **std::unordered_multiset** is an associative container that contains element that doesn't follow specific order
- Elements can be duplicated
- Elements cannot be modified (change value) to not corrupt the container
- Objects that are stored in unordered_multiset need to overload 'operator=='

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Associative Containers

- **std::unordered_multimap** is an associative container that stores key value pairs without sorting
- Must define a template specialization for std::hash with specific 'key' type if the 'key' type is not native type
- Element keys are not unique
- Objects that are stored in unordered_set need to overload 'operator=='
- The 'operator[]' and 'at()' function are **not** available for unordered_multimap

Iterators

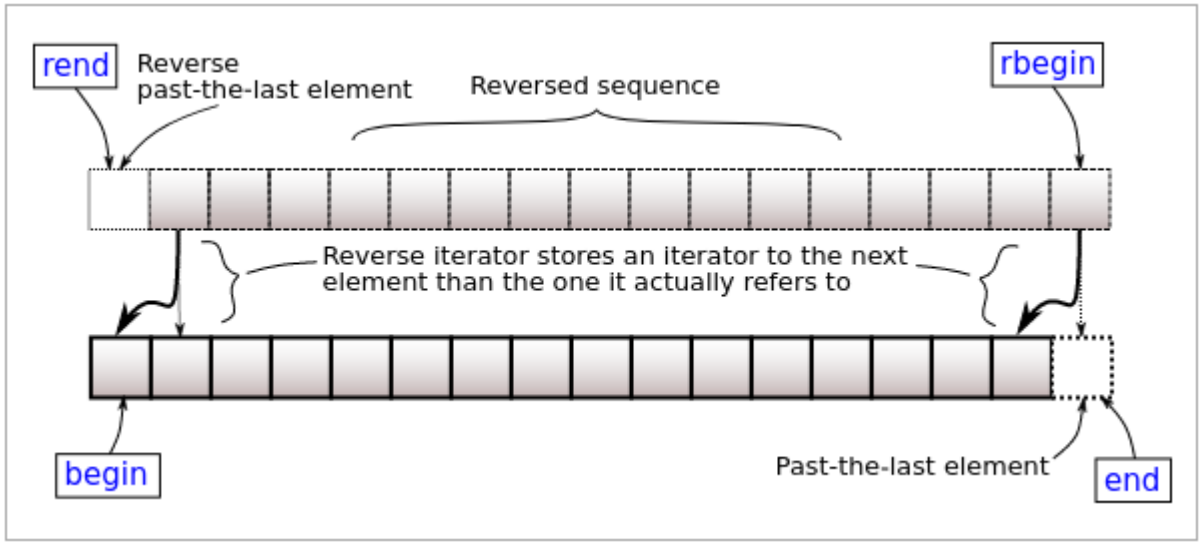
- Iterator is an object that allows parsing a container and provides access to the elements
- Iterator types
  - Forward iterator
  - Reverse iterator
  - Constant forward iterator
  - Constant reverse iterator
- Advantage of iterators: can parse the container with standard way without caring about container implementation, '++' operator for incrementation and '*' for accessing elements

Iterators

- Forward iterators
  - Iterator to get the container beginning «begin» (non-const) and «cbegin» (constant)
  - Iterator to get the container end «end» (non-const) and «cend» (constant)
- Reverse iterators
  - Iterator to get the container beginning «rbegin» (non-const) and «crbegin» (constant)
  - Iterator to get the container end «rend» (non-const) and «crend» (constant)

Iterators

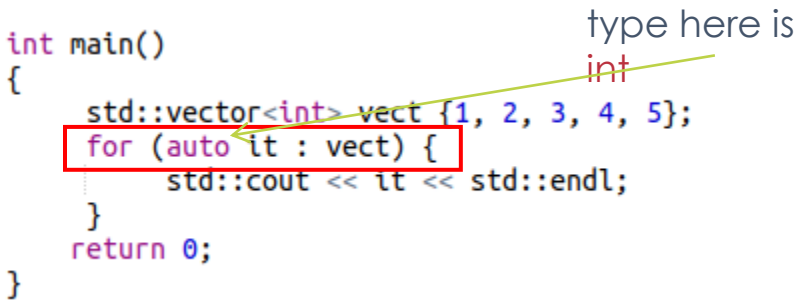
- Difference between forward and reverse iterators



- «end» and «rend» are always not part of container, but just gives reference to container ending

Iterators

- Range based loops



- Iterator invalidation: when an iterator is iterating on an element that has changed address or destroyed, we say that the iterator has been **invalidated**. So, the iterator is dangling and accessing it causes an undefined behavior:

```
v.erase(it); // erase the element currently being iterated over
// erase() invalidates iterators to the erased element (and subsequent elements)
// so iterator "it" is now invalidated
++it; // undefined behavior
```

Iterators

- Operators for iterators
  - operator* dereferencing the iterator returns the element value the iterator is pointing on
  - operator++ move iterator to point on next element
  - operator-- move iterator to point on next element (not always available for all containers)
  - operator== and operator!=
  - operator= assigning an iterator to another

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Iterators

Generally, to iterate through a container, we use the following pattern

```cpp
std::vector<int> vect {1, 2, 3, 4, 5};
// using while loop
std::vector<int>::iterator it1 = vect.begin();
while (it1 != vect.end()) {
     std::cout << *it1 << std::endl; // access element value
     (*it1)++; // increment value
     it1++; // increment iterator
}
// using for loop
for (auto it = vect.cbegin(); it != vect.cend(); ++it) {
     std::cout << *it << std::endl;
     // cannot change value since it is const iterator
     //(*it)++;
}
```

Forward iterator

```cpp
std::vector<int> vect {1, 2, 3, 4, 5};
// using while loop
std::vector<int>::reverse_iterator it1 = vect.rbegin();
while (it1 != vect.rend()) {
     std::cout << *it1 << std::endl; // access element value
     (*it1)++; // increment value
     it1++; // increment iterator
}
// using for loop
for (auto it = vect.crbegin(); it != vect.crend(); ++it) {
     std::cout << *it << std::endl;
     // cannot change value since it is const iterator
     //(*it)++;
}
```

Reverse iterator

Smart Pointers

- Smart pointers are classes that wraps pointers, so allocation and deallocation of pointer is done respectively in constructor and destructor. This follows the RAII paradigm.
- Available smart pointers in standard library
  - std::unique_ptr<T>: smart pointer with unique object ownership
  - std::shared_ptr<T>: smart pointer with shared object ownership
  - std::weak_ptr<T>: smart pointer that references an object managed by std::shared_ptr
  - std::auto_ptr<T>: smart pointer with strict object ownership (deprecated not to be used)

Smart Pointers

- std::unique_ptr<T> used to manage allocation and deallocation of object of type 'T' that is not shared by multiple object.
- Copy constructor of unique_ptr is deleted (can only be moved) so that ownership is moved as well
- Copy assignment is also deleted (can only use move assignment)
- In C++ 14, we use generally '*std::make_unique<T>(constructor_parameters)*' to create unique_pointer

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Smart Pointers

- std::unique_ptr<T>
- Returning std::unique_ptr from function
  - Function ‘createA()’ returns unique_ptr by value, if it is not assigned during call, unique_ptr will go out of scope. If it is assigned like in example, it will be moved to ‘ptr’ (since copy is deleted and return is an rvalue.
  - Generally, should not return unique_ptr by reference or pointer unless there is a reason (but it is doable like shown in examples)

```
std::unique_ptr<A> createA()
{
    return std::make_unique<A>();
}

int main()
{
    auto ptr{ createA() };
    // processing
    return 0;
}
```

Smart Pointers

- std::unique_ptr<T>
- Passing unique_ptr to function: generally, we have to pass it by value, since copy is disabled, move semantics should be used by calling 'std::move'
- Always be careful on object ownership, moved unique_ptr is no longer valid and may go out of scope and destroyed

```
void takeOwnership(std::unique_ptr<A> res)
{
} // the Resource is destroyed here

int main()
{
    auto ptr{ std::make_unique<A>() };
     // This doesn't work, need to use move semantics
     //    takeOwnership(ptr);
     // ok: use move semantics
    takeOwnership(std::move(ptr));
```


ACTIA®

Smart Pointers

- std::shared_ptr<T> is used to manage an object that will be shared by many users
- It won't be destroyed unless all shared instances go out of scope (it holds an internal counter that counts the number of shared instances and will stay alive until counter goes back to 0)
- Copy is allowed for shared_ptr, it will increment the local counter instead of creating new object (the object wrapped by shared_ptr remains the same)
- In C++ 14, we use generally '*std::make_shared<T>(constructor_parameters)*' to create shared_pointer (very recommended for better performance)

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Smart Pointers

- std::shared_ptr<T>
- It can be copied as well as moved, no restrictions
- Generally it is passed by value and returned by value
- A ‘unique_ptr’ can be converted to ‘shared_ptr’ using a special ‘shared_ptr’ constructor, in this case, the object of unique_ptr will be moved to shared_ptr

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Smart Pointers

- Issue with shared_ptr: circular dependency
- In this case, destructors of x1 and x2 are not called!!
- What happens ?
- When x1 goes out of scope (shared_ptr), it is already held by x2 and vice versa -> use_count is not 0 -> destructor is not called

```
class X
{
    std::shared_ptr<X> circular;
    std::string id;
public:
    X(const std::string& id) : id{id}
    {
        std::cout << id << " created" << std::endl;
    }
    virtual ~X()
    {
        std::cout << id << " destroyed" << std::endl;
    }
    void setCircular(std::shared_ptr<X> & c)
    {
        circular = c;
    }
};

int main()
{
    // circular reference issue
    auto x1 = std::make_shared<X>("X1");
    auto x2 = std::make_shared<X>("X2");
    x1->setCircular(x2);
    x2->setCircular(x1);
     return 0;
}
```

Smart Pointers

- std::weak_ptr<T> was designed to resolve the circular dependency issue
- It is just an observer of shared_ptr, not an owner
- It must be converted to shared_ptr using 'lock()' method to access the resource
- To solve previous circular dependency, 'circular' menber is weak_ptr (will just monitor and not holding the object) so the destructor is called

```
class X
{
    std::weak_ptr<X> circular;
    std::string id;
public:
    X(const std::string& id) : id{id}
    {
        std::cout << id << " created" << std::endl;
    }
    virtual ~X()
    {
        std::cout << id << " destroyed" << std::endl;
    }
    void setCircular(std::shared_ptr<X> & c)
    {
        circular = c;
    }
};
```

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Algorithm library

- Standard library comes with a set of predefined algorithms functions that facilitates some standard processing like searching, sorting, counting... that are applied generally to containers
- Three categories are provided
  - Inspectors: no modifications are brought to containers e.g. searching and counting elements
  - Mutators: that modify containers e.g. sorting, shuffling
  - Facilitators: generate result based valueof the data member
- Number of functions is huge, only most useful APIs will be seen here. Otherwise, it is

Algorithm library: non-modifying

- std::for_each : parse a range of container using iterators and perform unary operation on each element

```
int count{0};
std::list<int> ls {1, 3, 10, 25 , 5};
std::for_each(ls.cbegin(), ls.cend(), [&count](int& n){
   std::cout << "Element-" << count << ": " << n << std::endl;
   count++;
});
```

Using Lambdas

```
void print(int& n) {
        std::cout << "Element" << ": " << n << std::endl;
}

int main() {
     std::list<int> ls {1, 3, 10, 25 , 5};
     std::for_each(ls.begin(), ls.end(), print);
```

Using function

```
class Printer {
public:
     static int count;
     void operator()(const int& n) {
        std::cout << "Element-" << count << ": " << n << std::endl;
        count++;
     }
};
int Printer::count = 0;
int main() {
     std::list<int> ls {1, 3, 10, 25 , 5};
     std::for_each(ls.cbegin(), ls.cend(), Printer());
```

Using functors

Algorithm library: non-modifying

- std::find: returns an iterator to the first element equals to the provided element in arguments
- std::find_if: returns an iterator to the first element tested by the predicate which returns true whe

```
std::deque<int> dq {1, 3, 10, 25 , 5};
std::deque<int>::const_iterator it = std::find(dq.cbegin(), dq.cend(), 25);
if (it != dq.cend()) {
     std::cout << "Found " << *it << std::endl;
}
std::deque<int>::const_iterator itf = std::find_if(dq.cbegin(), dq.cend(), [](const int& n){
   return (n % 2 == 0);
});
if (itf != dq.cend()) {
     std::cout << "Found even " << *itf << std::endl;
}
```

Algorithm library: non-modifying

- std::search: find a range of elements inside a container and return an iterator to the first occurence
- std::find_end: find a range of elements inside a container and return an iterator to the last occurence

```
std::vector<A> vect {A(3), A(5), A(6), A(2), A(4), A(15), A(5), A(6), A(10), A(8)};
std::list<A> ls {A(5), A(6)};
std::vector<A>::iterator its = std::search(vect.begin(), vect.end(), ls.begin(), ls.end());
if (its != vect.end()) {
    std::cout << "Found range with search at position " << std::distance(vect.begin(), its) << std::endl;
}
its = std::find_end(vect.begin(), vect.end(), ls.begin(), ls.end());
if (its != vect.end()) {
    std::cout << "Found range with find_end at position " << std::distance(vect.begin(), its) << std::endl;
}
```

Algorithm library: non-modifying

- std::find_first_of: searches for the first occurence of an element in a sequence in a given container and returns an iterator to it

```
std::set<int> set {5, 6, 1, 8, 10, 19, 28, 4, 3};
std::list<int> find_list {19, 8};
std::cout << set;
std::set<int>::iterator itsS = std::find_first_of(set.begin(), set.end(), find_list.begin(), find_list.end());
if (itsS != set.end()) {
    std::cout << "Found element in set " << *itsS << " at position " << std::distance(set.begin(), itsS) << std::endl;
}
```

Algorithm library: non-modifying

- std::adjacent_find: searches the range of elements in order to find the first pair of adjacent consecutive elements. If such a pair is found, the iterator to the first element is returned.
- Needs 'operator==' overload for user defined types for default version or define binary predic

```
std::list<char> cls {'a','b', 'c', 'a', 'A', 'b','D', 'A', 'b', 'C', 'a', 'f'};
std::list<char>::iterator itlc = std::adjacent_find(cls.begin(), cls.end(), Comparer());
if (itlc != cls.end()) {
    std::cout << "Found adjacent elements " << *itlc << "/" << *(++itlc) << " at position "
              << std::distance(cls.begin(), --itlc) << std::endl;
}
```

Algorithm library: non-modifying

- std::count: returns the number of occurences of an element

```
std::list<char> cls {'a','b', 'c', 'a', 'A', 'b','D', 'A', 'b', 'C', 'a', 'f'};
std::size_t nb = std::count(cls.begin(), cls.end(), 'a');
std::cout << "Number of occurences of 'a' is " << nb << std::endl;
```

- std::count_if: returns the number of occurences of an element for which predicate returns true

```
std::list<char> cls {'a','b', 'c', 'a', 'A', 'b','D', 'A', 'b', 'C', 'a', 'f'};

std::size_t nb = std::count_if(cls.begin(), cls.end(), [](const char& c){
    return std::tolower(c) == 'a';
});
std::cout << "Number of occurences of 'a' or 'A' is " << nb << std::endl;
```

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Algorithm library: non-modifying

std::mismatch: compares two ranges of elements, and returns the iterators which point to the place where those sequences differ. If the ranges are identical, the function returns last iterator, and the corresponding iterator value from the second sequence.

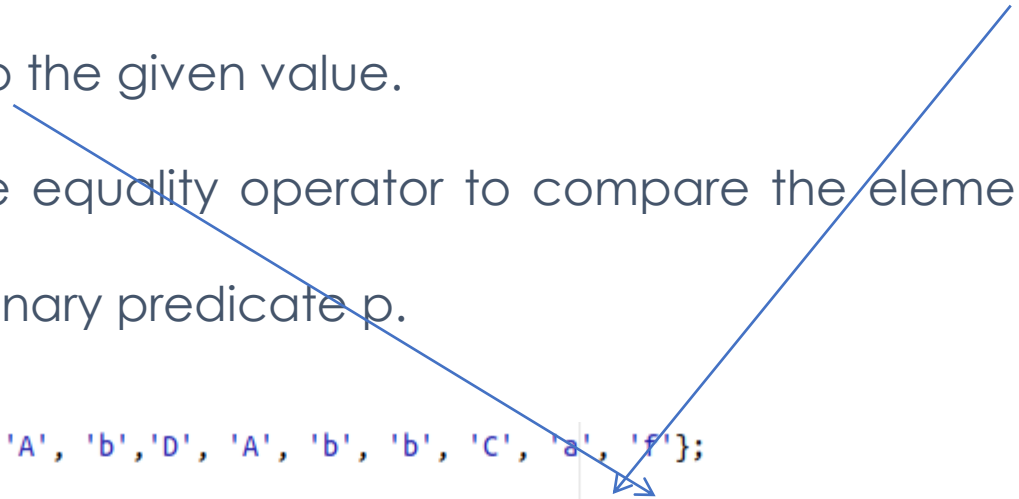
```
std::vector<int> vect_m {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
std::list<int> list_m {1, 2, 4, 6, 7, 3, 5, 9, 10, 8};
std::pair<std::vector<int>::iterator, std::list<int>::iterator> p1 = std::mismatch(vect_m.begin(), vect_m.end(), list_m.begin());
if (p1.first == vect_m.end() && p1.second == list_m.end())
{
    std::cout<<"Collections vect_m / list_m are equal!" << std::endl;
}
else
{
    std::cout << "Collections vect_m / list_m are NOT equal!" << std::endl;
    std::cout << "Mismatch at position: " << std::distance(vect_m.begin(), p1.first) << std::endl;
    std::cout << "First not matching element - vect_m: " << *(p1.first)<< " list_m: " << *(p1.second) << std::endl;
}
```

☐ Provisoire ☐ Public ☐ Interne ☑ Confidentiel ☐ Secret

ACTIA®

Algorithm library: non-modifying

- std::search_n: searches the range first, last for the first sequence of count identical elements, each equal to the given value.
- The first version uses the equality operator to compare the elements, while the second version uses the given binary predicate p.



```
std::list<char> cls {'a','b', 'c', 'a', 'A', 'b','D', 'A', 'b', 'b', 'C', 'a', 'f'};

std::list < int >::iterator found = std::search_n (cls.begin (), cls.end (), 2, 'b');

std::list < char >::iterator found_c = std::search_n (cls.begin (), cls.end (), 2, 'A', [] (const char& c1, const char& c2){
    return std::tolower(c1) == std::tolower(c2);
});
```

Algorithm library: modifying

- std::copy: copies elements from the range (first, last) to a new range, which starts with the iterator of destination range. After each element is copied, the target iterator is increased.
- This function doesn't add or insert values into the destination collection (the range); it only assigns values, so the destination range must have enough space allocated to it.
- The source and target ranges should not overlap in such a way that a particular element would be writ

```
std::vector<A> vect_a {A(1), A(5), A(4)};
std::deque<A> deq_a;
deq_a.resize(3, A(0));
std::copy(vect_a.begin(), vect_a.end(), deq_a.begin());
```

Algorithm library: modifying

- std::swap: perform the swapping of elements. After the invocation, the values of source and target are switched.
- std::swap_ranges: Exchange element between range [first, last) with range starting at first2
- std::iter_swap: given iterators are pointing to.

```
std::vector<A> vect_a {A(1), A(5), A(4)};
std::vector<A> vect_a1 {A(10), A(50), A(40), A(45)};
std::swap(vect_a, vect_a1);
auto it = vect_a.begin();
std::advance(it, 2);
std::swap_ranges(vect_a.begin(), it, vect_a1.begin());

std::iter_swap(vect_a.begin(), vect_a1.begin());
```

Algorithm library: modifying

- std::transform: works with unary and binary operations
  - For unary operation, apply processing done by this operation on range [first, last) and stores result in destination iterator. The return value should have same type as destination

```
Ret fun(const Type &a);
```

  - For binary operation, apply processing done by this operation on ranges [first, last) and first2 and stores result on destination iterator. The return value should have same type as destination. The operands should have same type as first and first2 iterators

```
Ret fun(const Type1 &a, const Type2 &b);
```

Algorithm library: modifying

- std::replace: replace fitting value with a new one in the given iterator range, it requires the operator== to be present in the class held by container

```
std::replace(source_vect1.begin(), source_vect1.end(), A(45), A(90));
```

- std::replace_if: replace fitting value (when predicate is returning true) with a new one in the given iterator range

```
std::replace_if(source_vect1.begin(), source_vect1.end(), [](const A& a){
    if (a.getX() == 90) {
        return true;
    }
    else {
        return false;
    }
}, A(100));
```

Algorithm library: modifying

- std::fill, std::fill_n: assign the same value value to all elements specified, either by range [first, last), or by the output iterator ofirst and the count of the elements to fill.
- Container must be allocated and have at least the size to fill

```
std::fill(list_fill.begin(), list_fill.end(), A(10));

std::fill_n(list_fill.begin(), 5, A(25));
```

Algorithm library: modifying

- std::generate: works like fill, but not using a unique value but a value got from a generator function
- Generator signature «Ret generator(void)» where return type is the same as the element held by the container to be filled

```
std::generate(vect_generate.begin(), vect_generate.end(), Generator());


std::generate_n(vect_generate.begin(), 20, Generator());
```

Algorithm library: modifying

- std::remove and std::remove_if: remove elements from collection
- First one remove elements of given value
- Second one remove elements where predicate returns true
- Both of them return an iterator to the new container end

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

String

- std::string is a class introduced in C++ to make string management easier (list of characters)
- It is a container named basic_string with ‘char’ as elements type (many analogies with containers management like iterators and APIs)
- Can be created using list of characters, copied or moved from another string
- The most similar container to string is vector, almost all member functions are the same

String

- Construction: below are different ways to create a string

```
std::string str; // empty string
std::string char_str { "This is a string" };
str = char_str; // copy assignment
std::string copy_str { str }; // copy constructor
std::string move_string { std::move(str)};
std::cout << "Moved string " << move_string << " and old one *" << str << "*" << std::endl;
std::string sub_string (copy_str, 4);
std::cout << "New string from 4 first character:" << sub_string << std::endl;
std::string sub_string2 (copy_str, 5, 2);
std::cout << "New string from 5 index 2 characters:" << sub_string2 << "*" << std::endl;
std::string from_numeric { std::to_string(2.0) };
std::cout << "From numeric " << from_numeric << std::endl;
```

String

- Length and capacity
  - '.length()' returns the number of characters inside the string
  - '.empty()' returns a boolean to check if the string is empty
  - '.capacity()' returns the number of characters that can be put without reallocation
  - '.reserve()' allocated memory for string without initialization

String

- Accessing string:
  - operator[] and '.at()' can be used to access characters at given position
  - '.c_str()' returns a pointer to constant buffer of string «const char*»
  - '.data()' returns a pointer to buffer of string «char*»

String

- Assignment:
  - '.assign()': overrides the containts of string (returns *this to chain calls)
  - operator=: equivalent to assign
  - '.append()': concatenates strings (returns *this to chain calls)
  - operator+=: equivalent to append

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

String

- Insertion, clear and subtring
  - '.insert()' used to insert data into string (return *this for chaining or an iterator to first inserted character)
  - '.clear()' to erase the string
  - '.substr()' to get a substring from another

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Date and time management

- For C++, standard library provides ‘chrono’ library that allows manipulating date and time in source code
- The ‘chrono’ library’ contains three main types of utilities:
  - clocks
  - time points
  - durations

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Date and time management

- Clocks: a clock consists of a starting timepoint named «epoch»
  - std::chrono::system_clock: represents sytem clock, non-monotonic clock (can be modified at any time) and can be mapped to C-style time
  - std::chrono::steady_clock: monotonic clock (that can be never modified), and it is most suitable for mesuring intervals (like time since last reboot, ...)
  - std::chrono::high_resolution_clock: a clock with smallest tick period, used for precision time mesurement
  - Refer to example for details on how to use clocks

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Multi-threading and threads synchronization

- Threads management and concurrency support
  - Create and manage threads
  - Atomic operations
  - Mutual exclusion
  - Condition variables
  - Futures and promises

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Multi-threading and threads synchronization

- Create and manage threads
  - 'std::thread' used to create standard threads. It allows to run multiple functions concurrently
  - Threads begin immediately upon construction
  - If Exception is thrown within thread, std::terminate is called
  - Promise and Future mechanism is used to communicate with caller (return value or handled exceptions)
  - Mutexes and atomic variables are used to manage cuncurrent access for shared data

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Multi-threading and threads synchronization

- Create and manage threads
  - Important: thread must be joined by calling 'join()' function (blocking until processing is finished) OR detached using 'detach()' function so that thread remains running separately untils self destruction (thread function exits)
  - If thread is detached, the thread handle is no longer valid

Multi-threading and threads synchronization

- Atomic variables
  - Race condition is a common issue faced on multi threading systems
  - It occurs when many threads try to access same resource at the same time
  - Atomic variables are created to be protected from data races
  - ‘std::atomic<T>’ where T is the type of atomic variable
  - Type aliases are defined in standard library like std::atomic_bool ~ std::atomic<bool>

Multi-threading and threads synchronization

- Mutual exclusion: is satisfied by mutexes:
  - std::mutex
  - std::timed_mutex
  - std::recursive_mutex
  - std::recursive_timed_mutex
  - std::shared_timed_mutex

Multi-threading and threads synchronization

- Mutual exclusion: '**std::mutex**' offers exclusive non-recursive ownership which means:
  - Calling thread owns the mutex since call of 'lock()' or 'try_lock' until calling unlock
  - 'lock()' is blocking function that returns when mutex is owned by the calling thread
  - 'try_lock()' returns immediately is the mutex is not available
  - When a thread owns the mutex, all other threads trying to own it are blocked in 'lock()' function call until the owner releases the mutex by calling 'unlock()'
  - std::mutex is neither copyable nor movable (it can be only be passed by reference or address)

Multi-threading and threads synchronization

- Mutual exclusion: '**std::mutex**'
  - 'std::lock_guard' can be used to automatically call 'lock()' when declared and 'unlock()' when out of scope
  - 'std::unique_lock' is a general purpose mutex wrapper (suits all mutexes types)
- The wrappers above can be used with 'std::mutex'

Multi-threading and threads synchronization

- Mutual exclusion: '**std::timed_mutex**' is same as 'std::mutex' but it offers timing duration to wait for the mutex ownership by 'try_lock_for()' and 'try_lock_until()'
  - Timed mutex is neither movable nor copyable
  - It is used when we we sometimes need to jump the lock of mutex after a period of time

Multi-threading and threads synchronization

- Mutual exclusion: '**std::recursive_mutex**' as all mutexes, it is used to protect shared data from beeing accessed at the same time
  - It offers exclusive and recursive ownership of mutex i.e. when a thread owns a recursive_mutex, all other thread will be blocked when calling lock() function
  - Recursivity is satisfied by the fact that lock() or try_lock() can be called many times in the owner thread and the mutex is release when unlock() is called as many as lock() called.

Multi-threading and threads synchronization

- Mutual exclusion: '**std::recursive_timed_mutex**' is a recursive mutex with try lock duration
  - Use try_lock_for() and try_lock_until() to manage lock waiting timeout
  - It is neither movable nor copyable

Multi-threading and threads synchronization

- Mutual exclusion: '**std::shared_timed_mutex**' is a new type of mutexes, introduced in C++14
  - Same as other mutexes, it aims to protect shared data from being accessed simultaneously by multiple threads
  - It has **two levels** of access
    - Exclusive: only one thread can own the mutex (using lock(), try_lock(), try_lock_for()...)
    - Shared: many threads can share ownership of mutex (using lock_shared(), try_lock_shared(), try_lock_shared_for()...)

Multi-threading and threads synchronization

- Condition variables: is a synchronization primitive used with std::mutex to block one or many threads until another one modifies shared variable and notifies one or all of them to continue (allow them to continue)
- Even shared variable can be atomic, it must be modified by owning the mutex to correctly publish the modification to waiting threads.
- If no shared variable needs to be modified, condition_variable can be used as synchronization tool between threads (some threads are waiting for a processing to be done (when calling notify_one or notify_all)

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Multi-threading and threads synchronization

- Steps for thread that wants to modify shared variable:
  - Acquire a mutex (get ownership)
  - Modify shared variable
  - call notify_one() or notify_all()
- Steps for threads that intend to wait on condition variable
  - Acquire **unique_lock** of mutex (same one used to protect shared variable)
  - Call wait(), wait_for() or wait_until()
  - Check if notified (in case of timeout) and continue processing

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Multi-threading and threads synchronization

- Futures and promises: provide a facility to provide data from a thread to another (or to the caller) and that can be acquired later asynchronously
- Future and promise are linked together: promise is used to set the data that will be passed and future generated from promise to be used to get data (in the future :D)
- The data held by promise is called ‘shared state’
- So, promise is used to set the shared state and generate future that will be used to get the shared state

Multi-threading and threads synchronization

- std::promise<T> where T is the type of shared state (can be void as well)
  - 'set_value()' used to set shared state value
  - 'set_exception()' used to set share state as exception
  - 'get_future()' returns the associated future
  - **Important**: Promise is intended to be used only once

Multi-threading and threads synchronization

- std::future<T> where T is the type of the shared state, must be the same as the type of promise
  - It can be retrieved from std::promise, std::async or std::packaged_task
  - Its shared state has to be in valid state before use (or undefined behavior)
  - A valid shared state means that the future is associated to promise or the 'get()' function has not been called to retrieve the shared state value
  - Can use wait functions to wait for shared state to have a value (set by promise)
  - 'get()' function wait until shared state is ready before returning
  - 'valid()' checks if the shared state is valid

Multi-threading and threads synchronization

- std::shared_future<T> is same as std::future, however it can be shared by many threads (unlike future which is not copyable but only movable)
- All shared_future are pointing to same shared state

Multi-threading and threads synchronization

- std::async: runs a function ansynchronously (like threads) and returns std::future having the type of return value of the function
- It can be launched immediately std::launch::async
- It can be launched when result is claimed (waiting for future result) using std::launch::deferred
- By default, launch is immediate

Multi-threading and threads synchronization

- std::packaged_task is a wrapper of callable target (function, lambdas, function object (functor), bind expression)
- The call of operator() of packaged_task is blocking, but it can be converted to std::thread to be run asyncronously (by calling std::move of packaged task as per thread function)
- The use of packaged_task is mostly to handle threads with return value (retrieved by 'get_future()' before moving packaed task to thread)

Back to Exceptions

- std::exception is the base class defined in standard library to manage exceptions. Many derived exceptions are used. For example
  - std::logic_error
    - std::invalid_argument
    - std::out_of_range ....
  - std::runtime_error
    - std::range_error
    - std:: overflow_error ...

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Back to Exceptions

- Other standard exceptions are defined and can be used, please refer to documentation
- Otherwise, user can define its own exception by creating a new one inheriting from std::exception and override the what() function to provide own exception

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

Files Management

- In standard library, files are considered as input, output or io streams:
  - std::ifstream (input file stream) derived from std::istream
  - std::ofstream (output file stream) derived from std::ostream
  - std::fstream (file stream) derived from std::iostream
- Means that files in this case are to be managed with stream operators << and >>
- Defined in file <fstream>
- See examples provided for more details

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Regular Expressions

Regular expression is a kind of language that allows pattern matching

Char	Description	Char	Description
.	Matches any character, ex: a.b matches ‘anb’, ‘aab’ ...	|	OR: ex: gr(e|a)y matches ‘gray’ or ‘grey’
?	the preceeding element is present zero or one occurence, ex: ab?c matches ‘abc’, ‘ac’	[c1-c2] [c1c2c3]	Matches range of chars between c1 and c2, a[a-zA-Z]b matches ‘avb’, ‘aEb’, axb’, ‘aXb’ .... Matches one of c1, c2 or c3, a[bcd]e matches ‘abe’ and ‘ace’
*	the preceeding element is present zero or more occurences, ex: ab*c matches ‘abc’, ‘ac’, ‘abbbbc’	[^c1-c2] [^c1c2]	Same as above, but any characher that is not specified between brackets []
+	the preceeding element is present one or more occurences, ex: ab+c matches ‘abc’, ‘abbbbc’	^ $	Matches the starting position in string Matches the end position in string
{n}	The preceeding element occurs exactly n times, ex ab{3}c mactches ‘abbbc’	()	Define groups. It won’t affect regex but can be used by regex_search to get position
{min,max}	The preceeding element occurs between min and max occurences.	[[:w:]] [[:alnum:]]	Matches an alphanumeric character including _ Many other characters are defined (see documentation)
{,max}	The preceeding element occurs up to max	[[:s:]]	White spaces
{min,}	The preceeding element occurs at least min times	\	Escape character

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Regular Expressions

- From previous table, we have to combine all tools to be able to get a pattern to match
- In standard library, the file <regex> represents the regex support library
- std::regex: a class that holds a regular expression
- std::regex_match: attempts to match a complete string to regex template
- std::regex_search: attempts to match a part of string to regex template
- regex_replace: attempts to replace matching regex pattern by a given string

ACTIA®
ENGINEERING SERVICES

AN ENTREPRENEURIAL SPIRIT

TO MOVE FORWARD TOGETHER

ACTIA Engineering Services

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®